

PROJECT REPORT ON

PHOENIXRV-32

**Design and Implementation of a 32-bit Single-Cycle RISC-V (RV32I)
Processor Using Verilog HDL**

Submitted in partial fulfilment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

prepared by

LEELA SHANMUKH YAGNEEK PATNALA



Department of Electronics and Communication Engineering

School of Electrical, Electronics and Communication Engineering

Vignan's Foundation for Science, Technology and Research (Deemed to be University)

Vadlamudi, Guntur, Andhra Pradesh-522213, India

May – 2026



CERTIFICATE

This is to certify that the project entitled **"PhoenixRV-32: Design and Implementation of a 32-bit Single-Cycle RISC-V (RV32I) Processor Using Verilog HDL"** is an independent hardware design project developed and implemented by **Leela Shanmukh yagneek Patnala**.

The processor was designed, coded, integrated, simulated, and functionally verified using Verilog HDL and open-source simulation tools. This report documents the architecture, implementation, verification, and results of the complete processor design.

Prepared By

Leela Shanmukh yagneek Patnala

DECLARATION

I, **Shanmukh Patnala**, hereby declare that this report entitled "**PhoenixRV-32: Design and Implementation of a 32-bit Single-Cycle RISC-V (RV32I) Processor Using Verilog HDL**" is my original work. The processor architecture, RTL implementation, simulation, verification, and documentation have been developed independently using Verilog HDL and publicly available technical references. Wherever external sources were consulted, appropriate acknowledgment has been provided.

Place: Tenali - Andhra Pradesh

Date: 1-07-2026

Signature

LEELA SHANMUKH YAGNEEK PATNALA

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Electronics and Communication Engineering, School of Electrical, Electronics and Communication Engineering, Vignan's Foundation for Science, Technology and Research (Deemed to be University) for providing the academic foundation and learning environment that helped me successfully complete this project.

I also acknowledge RISC-V International for providing the open-standard RISC-V Instruction Set Architecture (ISA), along with the developers of Icarus Verilog and EPWave, whose open-source tools were essential for the design, simulation, and functional verification of this processor. I am also thankful to the authors of technical books, research papers, and online resources that enhanced my understanding of processor architecture and Verilog HDL.

Finally, I would like to acknowledge my own dedication, continuous learning, and perseverance throughout the design, implementation, debugging, simulation, and documentation of PhoenixRV-32. This project has significantly strengthened my knowledge of computer architecture, RTL design, and digital system implementation.

Prepared By

Leela Shanmukh Yagneek Patnala

B.Tech – Electronics and Communication Engineering

ABSTRACT

PhoenixRV-32 is a 32-bit Single-Cycle processor based on the open-standard RISC-V Instruction Set Architecture (RV32I), designed and implemented using Verilog Hardware Description Language (HDL). The primary objective of this project is to develop a modular and functionally verified processor capable of executing the fundamental RV32I integer instruction set while providing a clear understanding of processor architecture and RTL design.

The processor architecture is divided into five major stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage has been implemented as an independent hardware module, including the Program Counter, Instruction Memory, Register File, Instruction Decoder, Immediate Generator, Control Unit, Arithmetic Logic Unit (ALU), ALU Control, Data Memory, and Write Back Unit. This modular design improves readability, simplifies debugging, and allows future expansion of the processor.

PhoenixRV-32 supports essential RV32I instructions, including arithmetic, logical, load, store, and branch operations such as ADD, SUB, AND, OR, XOR, SLT, LW, SW, and BEQ. Instruction execution is controlled using dedicated control signals generated by the Control Unit, while the ALU performs the required computations based on the instruction format.

The complete processor was functionally verified using Icarus Verilog for RTL simulation and EPWave for waveform analysis. A custom Verilog testbench and instruction memory file (instruction.mem) were developed to validate the execution of multiple instructions. Simulation results confirmed correct instruction fetching, decoding, execution, memory access, and write-back operations, demonstrating the correctness of the overall processor design.

PhoenixRV-32 serves as an educational and practical implementation of a RISC-V processor, providing a strong foundation for understanding computer architecture, digital system design, and RTL development. The modular architecture also enables future enhancements such as pipelining, hazard detection, cache memory, interrupt handling, FPGA implementation, and support for additional RISC-V instruction set extensions.

Keywords: RISC-V, RV32I, Verilog HDL, Single-Cycle Processor, Computer Architecture, RTL Design, FPGA, Digital Design, Processor Verification.

CONTENTS

Chapter No.	Description	Page No.
1	Introduction	1
2	Proposed Method or System	2
3	Working Principle	4
4	Results & Analysis	6
5	Advantages & Disadvantages	8
6	Applications	10
7	Future Scope	15
8	References	19

CHAPTER 1: INTRODUCTION

1.1 Introduction

Microprocessors are the heart of modern digital systems, powering devices ranging from smartphones and personal computers to automotive controllers and Internet of Things (IoT) devices. The continuous demand for efficient, scalable, and open hardware platforms has led to the widespread adoption of the RISC-V Instruction Set Architecture (ISA). Unlike proprietary processor architectures, RISC-V is an open standard that enables designers, researchers, and industries to develop customizable processors without licensing restrictions.

PhoenixRV-32 is a 32-bit Single-Cycle RISC-V processor designed and implemented using Verilog HDL. The processor follows the RV32I base integer instruction set and executes each instruction within a single clock cycle. The design emphasizes modular architecture, simplified datapath implementation, and functional verification. Every hardware block, including the Program Counter, Instruction Memory, Register File, Control Unit, Arithmetic Logic Unit (ALU), Data Memory, and Write Back Unit, has been independently designed and integrated to form a complete processor core.

The project demonstrates the complete processor design flow from architecture planning and RTL coding to simulation, waveform analysis, and functional verification. PhoenixRV-32 serves as a learning platform for computer architecture and digital hardware design while providing a scalable foundation for future enhancements such as pipelining, hazard detection, cache memory, and FPGA implementation.

1.2 Objectives:

The primary objectives of this project are:

- Design and implement a complete 32-bit Single-Cycle RISC-V processor.
- Support the RV32I base integer instruction set.
- Develop modular RTL blocks using Verilog HDL.
- Implement arithmetic, logical, load, store, and branch instructions.
- Verify processor functionality using simulation and waveform analysis.
- Build a synthesizable processor suitable for FPGA implementation.
- Create a scalable processor architecture for future enhancements.

1.3 Problem statement:

Designing a processor requires integrating multiple hardware components such as instruction memory, register files, ALUs, control logic, and memory interfaces into a unified datapath. Commercial processor architectures are often highly complex and difficult to understand for beginners. There is a need for a simplified yet functionally complete processor that demonstrates the complete instruction execution cycle while remaining modular, easy to debug, and suitable for academic learning and FPGA prototyping.

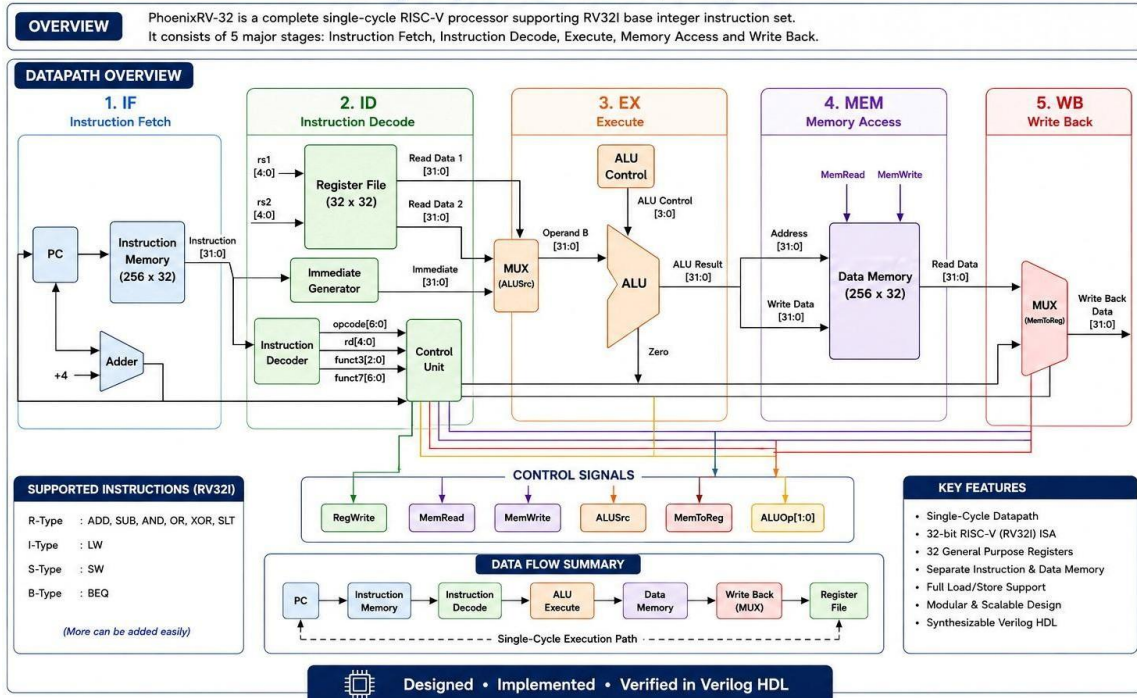
1.4 Scope of the Project

The scope of PhoenixRV-32 includes the design, implementation, simulation, and verification of a 32-bit Single-Cycle RISC-V processor. The processor supports the RV32I base instruction set, including arithmetic, logical, load, store, and branch operations. The project focuses on functional correctness rather than performance optimization and serves as a platform for future processor enhancements.

CHAPTER 2: PROCESSOR ARCHITECTURE

PhoenixRV-32 Architecture Overview

A Single-Cycle 32-bit RISC-V (RV32I) Processor



CHAPTER 2: PROCESSOR ARCHITECTURE

(Insert Architecture Overview Image here.)

2.1 Processor Overview

PhoenixRV-32 follows a Single-Cycle datapath architecture where every instruction completes execution in one clock cycle. The processor consists of five functional stages:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

Each stage is implemented as an independent hardware module, making the design modular and easy to maintain.

2.2 Instruction Fetch (IF)

The Instruction Fetch stage is responsible for retrieving instructions from instruction memory. The Program Counter (PC) stores the address of the current instruction and increments by four after each instruction to fetch the next sequential instruction. The instruction memory contains the program stored in hexadecimal format using the instruction.mem file.

2.3 Instruction Decode (ID)

The Instruction Decode stage extracts instruction fields such as opcode, destination register, source registers, and function codes. The Register File reads operand values, the Immediate Generator produces sign-extended immediate values, and the Control Unit generates control signals required by subsequent stages.

2.4 Execute (EX)

During the Execute stage, the Arithmetic Logic Unit (ALU) performs arithmetic and logical operations based on the control signals generated by the ALU Control Unit. Depending on the instruction type, the ALU operates on either register operands or immediate values. It also generates the Zero flag used for branch decisions.

2.5 Memory Access (MEM)

The Memory Access stage interfaces with the Data Memory module. Load instructions read data from memory, while store instructions write register values into memory. Memory operations are controlled using the MemRead and MemWrite control signals.

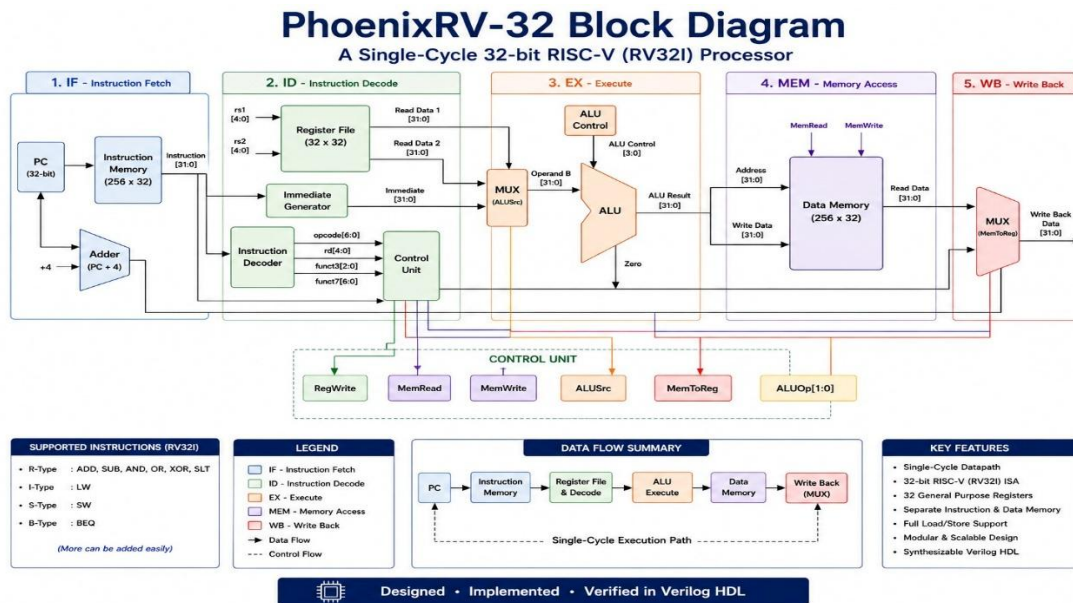
2.6 Write Back (WB)

The Write Back stage selects either the ALU result or memory data and writes the selected value back into the destination register using the Register File. The MemToReg control signal determines the source of the write-back data.

Processor Specifications:

Parameter	Value
Processor Name	PhoenixRV-32
Instruction Set Architecture (ISA)	RV32I
Processor Type	Single-Cycle RISC-V Processor
Architecture	Harvard Architecture
Data Width	32-bit
Address Width	32-bit
General Purpose Registers	32
Instruction Format	R, I, S and B Types
Instruction Memory	$256 \times 32\text{-bit}$
Data Memory	$256 \times 32\text{-bit}$
HardwareDescription Language	Verilog HDL
RTL Simulator	Icarus Verilog
Waveform Viewer	EPWave
Design Methodology	RTL Design
Verification Method	Functional Simulation & Waveform Analysis

CHAPTER 3: DESIGN METHODOLOGY



The development of PhoenixRV-32 followed a structured RTL design methodology. The processor architecture was first analyzed based on the RV32I instruction set. Individual hardware modules were designed independently using Verilog HDL and verified through dedicated simulations. After successful verification, all modules were integrated into a top-level processor core. Functional verification was then performed using a custom testbench, instruction memory initialization files, console output, and EPWave waveform analysis.

Design Flow:

Requirement Analysis → Architecture Design → RTL Coding → Module Testing → Integration → Simulation → Functional Verification

CHAPTER 4: MODULE DESCRIPTION

Program Counter:

Maintains the address of the current instruction and increments by four after every clock cycle.

Instruction Memory:

Stores program instructions and supplies them to the processor based on the current Program Counter value.

Decoder:

Extracts opcode, destination register, source registers, function codes, and other instruction fields.

Register File:

Contains 32 general-purpose registers with two read ports and one write port for operand access and result storage.

Immediate Generator:

Generates sign-extended immediate values for instructions requiring constant operands.

Control Unit:

Produces control signals such as RegWrite, MemRead, MemWrite, Branch, ALUSrc, MemToReg, and ALUOp based on the instruction opcode.

ALU Control:

Generates the appropriate ALU operation based on ALUOp, funct3, and funct7 fields.

Arithmetic Logic Unit:

Performs arithmetic and logical operations including ADD, SUB, AND, OR, XOR, and SLT while generating the Zero flag.

Data Memory:

Handles memory read and write operations for load and store instructions.

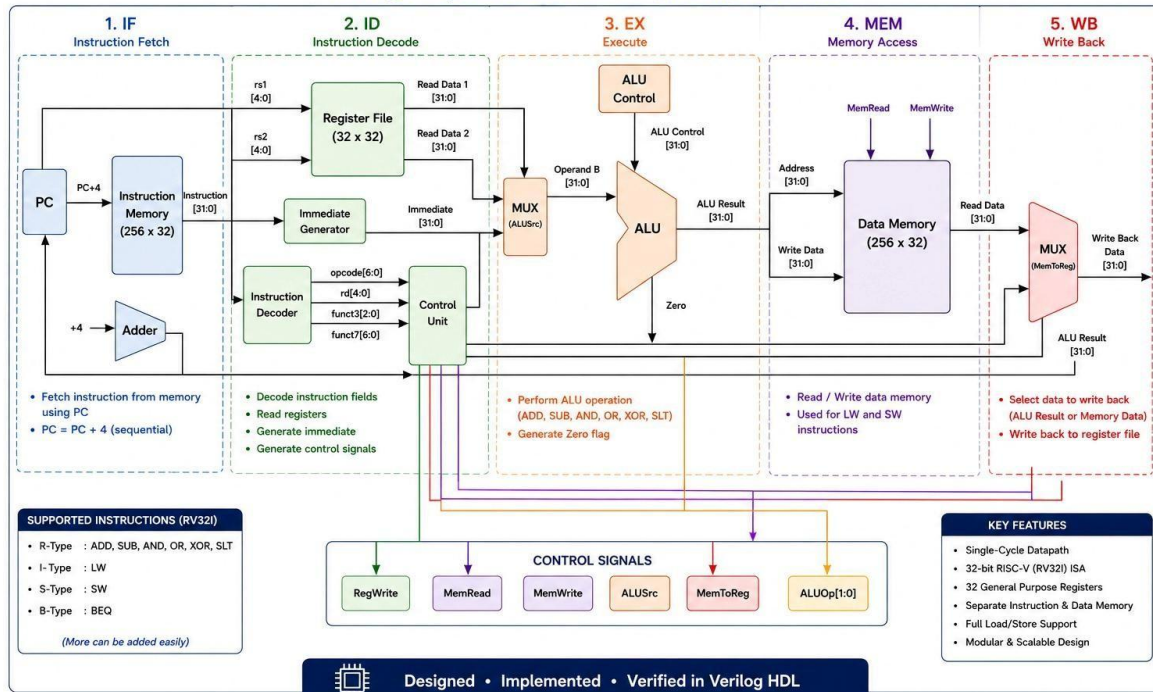
Write Back Unit:

Selects between ALU output and memory output for updating the destination register.

CHAPTER 5: PROCESSOR DATAPATH

PhoenixRV-32 Datapath

A Single-Cycle 32-bit RISC-V (RV32I) Processor



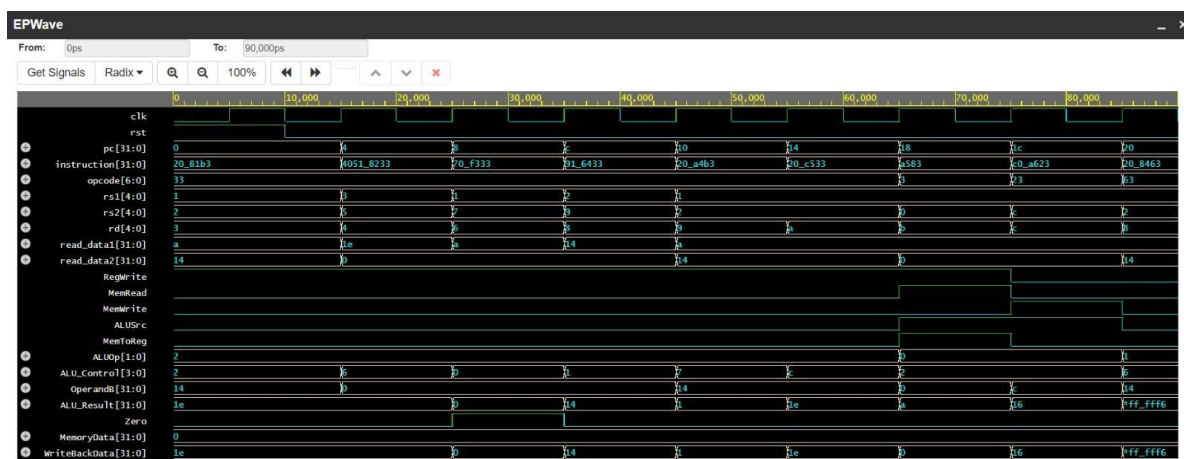
The datapath illustrates the complete flow of information inside PhoenixRV-32. Instructions are fetched from instruction memory, decoded to identify operands and control information, executed by the ALU, optionally access data memory, and finally write the result back to the register file. Multiplexers controlled by ALUSrc and MemToReg enable the processor to select appropriate operands and write-back sources.

CHAPTER 6: CONTROL SIGNALS

Signal	Description
RegWrite	Enables writing data into the register file.
MemRead	Enables reading data from memory.
MemWrite	Enables writing data into memory.
Branch	Indicates branch instruction execution.
ALUSrc	Selects between register data and immediate value as the ALU operand.
MemToReg	Selects whether ALU result or memory data is written back.
ALUOp	Specifies the type of ALU operation required.

CHAPTER 7: SIMULATION AND VERIFICATION

Functional verification of PhoenixRV-32 was performed using Icarus Verilog for simulation and EPWave for waveform visualization. A custom Verilog testbench generated the clock and reset signals while monitoring internal processor states. The instruction memory was initialized using the instruction.mem file containing hexadecimal machine instructions. Console output verified the execution of instructions, and waveform analysis confirmed correct timing relationships between the Program Counter, control signals, ALU outputs, memory operations, and write-back logic. The verification results demonstrated correct execution of supported RV32I instructions.



testbench sv

```
1 `timescale 1ns/1ps
2
3 module tb_rvcore32_top;
4
5 reg clk;
6 reg rst;
7
8 rvcore32_top DUT(
9
10 .clk(clk),
```

design sv

Instruction mem

```
1 `timescale 1ns/1ps
2
3 //-----
4 // program counter
5 //-----
6
7 module program_counter(
8     input clk,
9     input rst,
10    output reg [31:0] pc;
```

Log

Share

[2026-06-29 13:33:12 UTC] iverilog '-wall' '-g2012' design.sv testbench.sv && unbuffer vvp a.out

WARNING: design.sv:35: \$readmemh(instruction.mem): Not enough words in the file for the requested range [0:255].

VCD info: dumpfile rvcore32.vcd opened for output.

Time	PC	Instruction	RD	RD1	RD2	ALURes	MemData	WBData	RegWrite
0	00000000	002081b3	x3	10	20	30	0	30	1
15000	00000004	40518233	x4	30	0	30	0	30	1
25000	00000008	0070f333	x6	10	0	0	0	0	1
35000	0000000c	00916433	x8	20	0	20	0	20	1
45000	00000010	0020a4b3	x9	10	20	1	0	1	1
55000	00000014	0020c533	x10	10	20	30	0	30	1
65000	00000018	0000a583	x11	10	0	10	0	0	1
75000	0000001c	00c0a623	x12	10	0	22	0	22	0
85000	00000020	00208463	x8	10	20	4294967286	0	4294967286	0

testbench.sv:39: \$finish called at 90000 (1ps)

Finding VCD file...

./rvcore32.vcd

[2026-06-29 13:33:13 UTC] Opening EPWave...

Done

CHAPTER 8: RESULTS AND ANALYSIS

Simulation results confirm that PhoenixRV-32 correctly executes arithmetic, logical, load, store, and branch instructions. The Program Counter increments sequentially, the Instruction Decoder extracts instruction fields accurately, the Register File supplies operands correctly, and the ALU produces expected outputs. Memory operations and write-back logic were also verified using console outputs and waveform analysis. **The modular architecture simplified debugging and ensured functional correctness of each processor stage.**

CHAPTER 9: ADVANTAGES AND LIMITATIONS

Advantages

- Open-standard RV32I architecture
- Modular and scalable RTL implementation
- Simple single-cycle datapath
- Easy to understand and verify
- Synthesizable Verilog HDL design
- Suitable for FPGA implementation
- Strong educational value for processor design and computer architecture

Limitations

- Single-cycle architecture results in a longer clock period
- No instruction pipelining
- No cache memory
- No branch prediction
- No hazard detection or forwarding
- Limited to the RV32I base instruction set
- No interrupt or exception handling

CHAPTER 10: APPLICATIONS

PhoenixRV-32 can be used in computer architecture education, RTL design training, FPGA prototyping, embedded systems research, processor verification, and digital design laboratories. Its modular structure also makes it suitable as a foundation for developing advanced processors with pipelining, cache memory, and ISA extensions.

CHAPTER 11: FUTURE ENHANCEMENTS

Future improvements to PhoenixRV-32 may include implementing a five-stage pipelined architecture, hazard detection and forwarding units, branch prediction, cache memory, UART communication, timer interrupts, CSR registers, the RV32M multiplication and division extension, FPGA deployment on Xilinx Artix-7 devices, and additional optimization for higher performance and lower power consumption.

CHAPTER 12: CONCLUSION

PhoenixRV-32 successfully demonstrates the complete design and verification of a 32-bit Single-Cycle RISC-V processor based on the RV32I instruction set. The processor integrates instruction fetch, decode, execution, memory access, and write-back stages into a modular architecture capable of executing arithmetic, logical, load, store, and branch instructions. Functional verification through simulation and waveform analysis confirms the correctness of the datapath and control logic. The project provides a solid foundation for understanding processor architecture and RTL design while offering significant opportunities for future enhancements such as pipelining, cache integration, and FPGA implementation.

REFERENCES

1. RISC-V User-Level ISA Specification, RISC-V International.
2. David A. Patterson and John L. Hennessy, *Computer Organization and Design: RISC-V Edition*.
3. Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*.
4. Icarus Verilog Documentation.
5. EPWave User Documentation.
6. IEEE Standard for Verilog Hardware Description Language (IEEE 1364).
7. AMD Vivado Design Suite User Guide.